

汽车销售管理系统（SSM + Vue3）

day03-车辆品牌完整CRUD与前端管理-开发文档

预期成果：

- 后端车辆、品牌增删改查接口全部完成（Service层+Controller层）
- 前端车辆列表页具备新增、编辑、删除功能
- 品牌下拉框从后端动态加载（非硬编码）
- 车辆管理模块完整可用，含业务校验
- 重复校验规则为“品牌+车型+配置”三者组合唯一

一、今日任务概览

步骤	任务	负责人建议
1	完善 CarService 接口和 CarServiceImpl 实现类	后端同学
2	创建 BrandService 接口和 BrandServiceImpl 实现类	后端同学
3	完善 CarController 和 BrandController（注入Service）	后端同学
4	在Service层添加业务校验	后端同学
5	前端封装 car.js 和 brand.js API	前端同学
6	前端实现新增/编辑弹窗、删除功能（品牌下拉动态加载）	前端同学
7	Postman测试后端接口	全体
8	前后端联调测试	全体
9	Git提交与合并	全体

二、后端开发：车辆Service层

2.1 完善 CarService 接口

在 com.sales.service 包下编写 CarService.java：

```
package com.sales.service;

import com.sales.entity.Car;
import com.github.pagehelper.PageInfo;

import java.math.BigDecimal;
```

```

/**
 * 车辆管理业务接口
 */
public interface CarService {
    /**
     * 分页查询车辆（含品牌、价格筛选）
     */
    PageInfo<Car> findByPage(int pageNum, int pageSize, Integer brandId,
        BigDecimal minPrice, BigDecimal maxPrice);

    /**
     * 新增车辆
     */
    void add(Car car);

    /**
     * 修改车辆
     */
    void update(Car car);

    /**
     * 删除车辆
     */
    void delete(Integer id);

    /**
     * 根据ID查询车辆
     */
    Car findById(Integer id);
}

```

2.2 完善 CarService 实现类

在 `com.sales.service.impl` 包下编写 `CarServiceImpl.java`：

```

package com.sales.service.impl;

import com.github.pagehelper.PageHelper;
import com.github.pagehelper.PageInfo;
import com.sales.entity.Car;
import com.sales.entity.CarExample;
import com.sales.exception.BusinessException;
import com.sales.mapper.CarMapper;
import com.sales.service.CarService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.util.List;

@Service
@Transactional(rollbackFor = Exception.class)
public class CarServiceImpl implements CarService {

```

```

@Autowired
private CarMapper carMapper;

@Override
public PageInfo<Car> findByPage(int pageNum, int pageSize, Integer brandId,
BigDecimal minPrice, BigDecimal maxPrice) {
    // 1. 启动分页
    PageHelper.startPage(pageNum, pageSize);

    // 2. 构建查询条件（使用 MBG 生成的 Example）
    CarExample example = new CarExample();
    CarExample.Criteria criteria = example.createCriteria();

    if (brandId != null) {
        criteria.andBrandIdEqualTo(brandId);
    }
    if (minPrice != null) {
        criteria.andPriceGreaterThanOrEqualTo(minPrice);
    }
    if (maxPrice != null) {
        criteria.andPriceLessThanOrEqualTo(maxPrice);
    }

    // 3. 执行查询（按创建时间倒序，保持列表一致性）
    example.setOrderByClause("id DESC");
    List<Car> list = carMapper.selectByExample(example);

    // 4. 返回分页对象
    return new PageInfo<>(list);
}

@Override
public void add(Car car) {
    // 1. 参数校验
    if (car.getBrandId() == null) {
        throw new BusinessException("品牌不能为空");
    }
    if (car.getModel() == null || car.getModel().trim().isEmpty()) {
        throw new BusinessException("车型不能为空");
    }
    if (car.getConfig() == null || car.getConfig().trim().isEmpty()) {
        throw new BusinessException("配置不能为空");
    }
    if (car.getPrice() == null) {
        throw new BusinessException("价格不能为空");
    }
    if (car.getPrice().compareTo(BigDecimal.ZERO) <= 0) {
        throw new BusinessException("价格必须大于0");
    }

    // 2. 检查车型是否已存在（同一品牌+车型+配置组合不能重复）
    CarExample example = new CarExample();
    example.createCriteria()
        .andBrandIdEqualTo(car.getBrandId())
        .andModelEqualTo(car.getModel().trim())
        .andConfigEqualTo(car.getConfig().trim());
    List<Car> existList = carMapper.selectByExample(example);
}

```

```

        if (!existList.isEmpty()) {
            throw new BusinessException("该品牌下已存在相同车型和配置的车辆");
        }

        // 3. 库存默认0
        if (car.getStock() == null) {
            car.setStock(0);
        }

        // 4. 插入数据库
        carMapper.insertSelective(car);
    }

    @Override
    public void update(Car car) {
        // 1. 校验ID
        if (car.getId() == null) {
            throw new BusinessException("车辆ID不能为空");
        }

        // 2. 检查车辆是否存在
        Car existCar = carMapper.selectByPrimaryKey(car.getId());
        if (existCar == null) {
            throw new BusinessException("车辆不存在");
        }

        // 3. 判断品牌、车型、配置是否真正发生变化
        boolean brandChanged = car.getBrandId() != null &&
            !car.getBrandId().equals(existCar.getBrandId());
        boolean modelChanged = car.getModel() != null &&
            !car.getModel().trim().equals(existCar.getModel());
        boolean configChanged = car.getConfig() != null &&
            !car.getConfig().trim().equals(existCar.getConfig());

        // 4. 只有品牌、车型、配置发生变化时才校验重复
        if (brandChanged || modelChanged || configChanged) {
            Integer finalBrandId = car.getBrandId() != null ? car.getBrandId() :
            existCar.getBrandId();
            String finalModel = car.getModel() != null ? car.getModel().trim() :
            existCar.getModel();
            String finalConfig = car.getConfig() != null ?
            car.getConfig().trim() : existCar.getConfig();

            CarExample example = new CarExample();
            example.createCriteria()
                .andBrandIdEqualTo(finalBrandId)
                .andModelEqualTo(finalModel)
                .andConfigEqualTo(finalConfig)
                .andIdNotEqualTo(car.getId()); // 排除自己
            List<Car> existList = carMapper.selectByExample(example);
            if (!existList.isEmpty()) {
                throw new BusinessException("该品牌下已存在相同车型和配置的车辆");
            }
        }

        // 5. 更新
        carMapper.updateByPrimaryKeySelective(car);
    }

```

```

@Override
public void delete(Integer id) {
    // 1. 检查车辆是否存在
    Car existCar = carMapper.selectByPrimaryKey(id);
    if (existCar == null) {
        throw new BusinessException("车辆不存在");
    }

    // 2. 检查是否有关联的预约或订单（后续完善）
    // 3. 删除
    carMapper.deleteByPrimaryKey(id);
}

@Override
public Car findById(Integer id) {
    return carMapper.selectByPrimaryKey(id);
}
}

```

2.3 完善 CarController

编写 `CarController.java`，注入 `CarService`：

```

package com.sales.controller;

import com.github.pagehelper.PageInfo;
import com.sales.common.PageResult;
import com.sales.common.Result;
import com.sales.entity.Car;
import com.sales.service.CarService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;

@RestController
@RequestMapping("/car")
public class CarController {

    @Autowired
    private CarService carService;

    @GetMapping("/list")
    public Result list(@RequestParam(defaultValue = "1") int pageNum,
                      @RequestParam(defaultValue = "10") int pageSize,
                      @RequestParam(required = false) Integer brandId,
                      @RequestParam(required = false) BigDecimal minPrice,
                      @RequestParam(required = false) BigDecimal maxPrice) {

        // 调用 service 层
        PageInfo<Car> pageInfo = carService.findByPage(pageNum, pageSize,
        brandId, minPrice, maxPrice);
    }
}

```

```

        return Result.success(new PageResult(pageInfo.getTotal(),
pageInfo.getList()));
    }

    @PostMapping("/add")
    public Result add(@RequestBody Car car) {
        carService.add(car);
        return Result.success(car);
    }

    @PutMapping("/update")
    public Result update(@RequestBody Car car) {
        carService.update(car);
        return Result.success(car);
    }

    @DeleteMapping("/delete/{id}")
    public Result delete(@PathVariable Integer id) {
        carService.delete(id);
        return Result.success("删除成功");
    }

    @GetMapping("/{id}")
    public Result findById(@PathVariable Integer id) {
        return Result.success(carService.findById(id));
    }
}

```

关键点：Controller 只负责接收请求、调用Service、返回结果，不写业务逻辑。

三、后端开发：品牌Service层

3.1 创建 BrandService 接口

在 `com.sales.service` 包下创建 `BrandService.java`：

```

package com.sales.service;

import com.sales.entity.Brand;

import java.util.List;

public interface BrandService {
    List<Brand> findAll();
    void add(Brand brand);
    void update(Brand brand);
    void delete(Integer id);
    Brand findById(Integer id);
}

```

3.2 创建 BrandService 实现类

在 `com.sales.service.impl` 包下创建 `BrandServiceImpl.java` :

```
package com.sales.service.impl;

import com.sales.entity.Brand;
import com.sales.entity.BrandExample;
import com.sales.exception.BusinessException;
import com.sales.mapper.BrandMapper;
import com.sales.service.BrandService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional(rollbackFor = Exception.class)
public class BrandServiceImpl implements BrandService {

    @Autowired
    private BrandMapper brandMapper;

    @Override
    public List<Brand> findAll() {
        BrandExample example = new BrandExample();
        example.setOrderByClause("id ASC");
        return brandMapper.selectByExample(example);
    }

    @Override
    public void add(Brand brand) {
        if (brand.getName() == null || brand.getName().trim().isEmpty()) {
            throw new BusinessException("品牌名称不能为空");
        }

        // 检查品牌是否已存在
        BrandExample example = new BrandExample();
        example.createCriteria().andNameEqualTo(brand.getName().trim());
        List<Brand> existList = brandMapper.selectByExample(example);
        if (!existList.isEmpty()) {
            throw new BusinessException("品牌已存在");
        }

        brandMapper.insertSelective(brand);
    }

    @Override
    public void update(Brand brand) {
        if (brand.getId() == null) {
            throw new BusinessException("品牌ID不能为空");
        }

        Brand existBrand = brandMapper.selectByPrimaryKey(brand.getId());
        if (existBrand == null) {
            throw new BusinessException("品牌不存在");
        }
    }
}
```

```

        // 检查名称是否被其他品牌占用
        if (brand.getName() != null) {
            BrandExample example = new BrandExample();
            example.createCriteria()
                .andNameEqualTo(brand.getName().trim())
                .andIdNotEqualTo(brand.getId());
            List<Brand> existList = brandMapper.selectByExample(example);
            if (!existList.isEmpty()) {
                throw new BusinessException("品牌名称已被占用");
            }
        }

        brandMapper.updateByPrimaryKeySelective(brand);
    }

    @Override
    public void delete(Integer id) {
        Brand existBrand = brandMapper.selectByPrimaryKey(id);
        if (existBrand == null) {
            throw new BusinessException("品牌不存在");
        }
        // TODO: 检查是否有关联的车辆
        brandMapper.deleteByPrimaryKey(id);
    }

    @Override
    public Brand findById(Integer id) {
        return brandMapper.selectByPrimaryKey(id);
    }
}

```

3.3 创建 BrandController

```

package com.sales.controller;

import com.sales.common.Result;
import com.sales.entity.Brand;
import com.sales.service.BrandService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/brand")
public class BrandController {

    @Autowired
    private BrandService brandService;

    @GetMapping("/list")
    public Result list() {
        List<Brand> list = brandService.findAll();
        return Result.success(list);
    }
}

```



```

@PostMapping("/add")
public Result add(@RequestBody Brand brand) {
    brandService.add(brand);
    return Result.success(brand);
}

@PutMapping("/update")
public Result update(@RequestBody Brand brand) {
    brandService.update(brand);
    return Result.success(brand);
}

@DeleteMapping("/delete/{id}")
public Result delete(@PathVariable Integer id) {
    brandService.delete(id);
    return Result.success("删除成功");
}
}

```

四、前端开发：API封装

4.1 完善 src/api/car.js

```

import request from '../utils/request'

// 分页查询车辆列表
export function getCarList(params) {
    return request({
        url: '/car/list',
        method: 'get',
        params
    })
}

// 新增车辆
export function addCar(data) {
    return request({
        url: '/car/add',
        method: 'post',
        data
    })
}

// 修改车辆
export function updateCar(data) {
    return request({
        url: '/car/update',
        method: 'put',
        data
    })
}

// 删除车辆

```

```
export function deleteCar(id) {
  return request({
    url: `/car/delete/${id}`,
    method: 'delete'
  })
}
```

4.2 创建 src/api/brand.js

```
import request from '../utils/request'

// 查询所有品牌
export function getBrandList() {
  return request({
    url: '/brand/list',
    method: 'get'
  })
}
```

五、前端开发：车辆列表页面（完整版）

修改 `src/views/car/CarList.vue`

5.1 模板部分 (template)

1、品牌修改为动态获取：

```
<el-form-item label="品牌">
  <el-select
    v-model="searchForm.brandId"
    placeholder="请选择品牌"
    clearable
    style="width:150px;"
  >
    <el-option
      v-for="item in brandList"
      :key="item.id"
      :label="item.name"
      :value="item.id"
    />
  </el-select>
</el-form-item>
```

2、搜索区添加个“新增车辆”

```
<el-form-item>
  <el-button type="primary" @click="search">搜索</el-button>
  <el-button @click="reset">重置</el-button>
  <el-button type="primary" @click="handleAdd">+ 新增车辆</el-button>
</el-form-item>
```

3、展示表格中品牌ID栏修改为品牌栏

```
<el-table-column label="品牌" width="120">
  <template #default="{ row }">
    {{ getBrandName(row.brandId) }}
  </template>
</el-table-column>
```

4、展示表格中增加操作栏

```
<el-table-column label="操作" width="180">
  <template #default="{ row }">
    <el-button type="primary" size="small" @click="handleEdit(row)">编辑
  </el-button>
    <el-button type="danger" size="small" @click="handleDelete(row.id)">删除
  </el-button>
  </template>
</el-table-column>
```

5、新增/编辑弹窗

```
<!-- 新增/编辑弹窗 -->
<el-dialog v-model="dialogVisible" :title="dialogTitle" width="500px">
  <el-form :model="formData" :rules="rules" ref="formRef" label-width="120px">
    <el-form-item label="品牌" prop="brandId">
      <el-select v-model="formData.brandId" placeholder="请选择品牌"
style="width:100%;">
        <el-option
          v-for="item in brandList"
          :key="item.id"
          :label="item.name"
          :value="item.id"
        />
      </el-select>
    </el-form-item>
    <el-form-item label="车型" prop="model">
      <el-input v-model="formData.model" placeholder="请输入车型" />
    </el-form-item>
    <el-form-item label="配置" prop="config">
      <el-input v-model="formData.config" placeholder="请输入配置版本" />
    </el-form-item>
    <el-form-item label="价格(万元)" prop="price">
      <el-input v-model="formData.price" placeholder="请输入价格" />
    </el-form-item>
    <el-form-item label="库存" prop="stock">
      <el-input v-model="formData.stock" placeholder="请输入库存数量" />
    </el-form-item>
  </el-form>
  <template #footer>
    <el-button @click="dialogVisible = false">取消</el-button>
    <el-button type="primary" @click="submitForm">确定</el-button>
  </template>
</el-dialog>
```

5.2 脚本部分 (script setup)

```
<script setup>
import { reactive, ref, onMounted } from 'vue'
import { getCarList, addCar, updateCar, deleteCar } from '../api/car'
import { getBrandList } from '../api/brand'
import { ElMessage, ElMessageBox } from 'element-plus'

// ----- 搜索与列表 -----
const searchForm = reactive({
  brandId: '',
  minPrice: '',
  maxPrice: ''
})
const tableData = ref([])
const total = ref(0)
const pageNum = ref(1)
const pageSize = ref(10)

// ----- 品牌列表（动态加载） -----
const brandList = ref([])

// 加载品牌列表
const loadBrands = async () => {
  try {
    const res = await getBrandList()
    if (res.code === 200) {
      brandList.value = res.data || []
    }
  } catch (e) {
    console.error('加载品牌失败', e)
  }
}

// 根据品牌ID获取品牌名称
const getBrandName = (brandId) => {
  const brand = brandList.value.find(item => item.id === brandId)
  return brand ? brand.name : '未知品牌'
}

// ----- 加载车辆列表 -----
const loadData = async () => {
  try {
    const params = {
      pageNum: pageNum.value,
      pageSize: pageSize.value,
      brandId: searchForm.brandId || undefined,
      minPrice: searchForm.minPrice || undefined,
      maxPrice: searchForm.maxPrice || undefined
    }
    const res = await getCarList(params)
    if (res.code === 200) {
      tableData.value = res.data.rows || []
      total.value = res.data.total || 0
    }
  } catch (e) {
    console.error('加载失败', e)
  }
}
```

```

      ElMessage.error('加载车辆列表失败')
    }
  }

const search = () => {
  pageNum.value = 1
  loadData()
}

const reset = () => {
  searchForm.brandId = ''
  searchForm.minPrice = ''
  searchForm.maxPrice = ''
  search()
}

// ----- 新增/编辑弹窗 -----
const dialogVisible = ref(false)
const dialogTitle = ref('新增车辆')
const isEdit = ref(false)
const formRef = ref(null)

const formData = reactive({
  id: null,
  brandId: '',
  model: '',
  config: '',
  price: '',
  stock: ''
})

const resetFormData = () => {
  formData.id = null
  formData.brandId = ''
  formData.model = ''
  formData.config = ''
  formData.price = ''
  formData.stock = ''
}

// 表单校验规则
const rules = {
  brandId: [{ required: true, message: '请选择品牌', trigger: 'change' }],
  model: [
    { required: true, message: '请输入车型', trigger: 'blur' },
    { min: 2, max: 50, message: '车型名称长度在2-50个字符', trigger: 'blur' }
  ],
  price: [
    { required: true, message: '请输入价格', trigger: 'blur' },
    { pattern: /^\\d+(\\.\\d{1,2})?$/, message: '请输入有效的价格（最多两位小数）',
      trigger: 'blur' }
  ]
}

// 打开新增弹窗
const handleAdd = () => {
  dialogTitle.value = '新增车辆'
  isEdit.value = false

```

```

    resetFormData()
    dialogVisible.value = true
    // 清除校验状态
    formRef.value?.clearValidate()
  }

  // 打开编辑弹窗
  const handleEdit = (row) => {
    dialogTitle.value = '编辑车辆'
    isEdit.value = true
    Object.assign(formData, row)
    dialogVisible.value = true
    formRef.value?.clearValidate()
  }

  // 提交表单
  const submitForm = async () => {
    try {
      await formRef.value.validate()

      let res
      if (isEdit.value) {
        res = await updateCar(formData)
      } else {
        res = await addCar(formData)
      }

      if (res.code === 200) {
        ElMessage.success(isEdit.value ? '修改成功' : '新增成功')
        dialogVisible.value = false
        loadData() // 刷新列表
      } else {
        ElMessage.error(res.message || (isEdit.value ? '修改失败' : '新增失败'))
      }
    } catch (error) {
      // 网络异常或表单校验失败
      if (error !== false) {
        console.error('提交失败', error)
        ElMessage.error('操作失败，请检查网络')
      }
    }
  }

  // ----- 删除 -----
  const handleDelete = (id) => {
    ElMessageBox.confirm('确定要删除该车辆吗?', '提示', {
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'warning'
    }).then(async () => {
      try {
        await deleteCar(id)
        ElMessage.success('删除成功')
        loadData()
      } catch (e) {
        ElMessage.error('删除失败')
      }
    }).catch(() => {})
  }

```

```
}

// ----- 生命周期 -----
onMounted(() => {
  loadBrands()
  loadData()
})
</script>
```

六、联调测试

6.1 用 Postman 测试后端接口

方法	URL	请求体	预期结果
GET	/car/list? pageNum=1&pageSize=10	无	返回车辆分 页数据
POST	/car/add	{"brandId":1,"model":"Model s","price":68.99,"stock":5}	新增成功, 返回车辆对 象
PUT	/car/update	{"id":1,"price":22.99}	修改成功
DELETE	/car/delete/1	无	删除成功
GET	/brand/list	无	返回所有品 牌列表
POST	/brand/add	{"name":"蔚来"}	新增成功

6.2 完整前端流程测试

1. 进入车辆管理页面 → 品牌下拉框自动加载品牌列表（非硬编码）
2. 选择品牌、价格区间 → 点击搜索 → 列表刷新
3. 点击“新增” → 弹窗 → 选择品牌（动态加载）、填写信息 → 提交 → 列表刷新
4. 点击“编辑” → 弹窗回显数据 → 修改 → 提交 → 列表刷新
5. 点击“删除” → 确认弹窗 → 删除 → 列表刷新

6.3 异常场景测试

- 新增时品牌、车型、配置、价格不填 → 提示校验错误
- 新增时输入已存在的品牌+车型+配置 → 后端返回“该品牌下已存在相同车型和配置的车辆”
- 价格输入负数或字母 → 前端校验拦截
- 编辑时只改库存或价格 → 不触发重复校验，直接更新成功

七、常见问题与解决方案

问题	可能原因	解决方法
品牌下拉框为空	<code>loadBrands()</code> 未被调用	检查 <code>onMounted</code> 中是否调用了 <code>loadBrands()</code>
新增后列表不刷新	提交成功未调用 <code>loadData()</code>	在 <code>submitForm</code> 成功后调用 <code>loadData()</code>
新增时提示“品牌不能为空”	前端未选择品牌	检查 <code>el-select</code> 的 <code>v-model</code> 绑定是否正确
编辑时数据未回显	<code>Object.assign</code> 用法错误	使用 <code>Object.assign(formData, row)</code> 或逐个赋值
删除报“车辆不存在”	车辆已被删除，页面未刷新	删除成功后调用 <code>loadData()</code>
Postman测试返回500	Service层逻辑异常	查看IDEA控制台错误日志，定位具体行号
事务不回滚	Service方法中异常被try-catch吞掉	直接抛出异常，不捕获

八、Git提交与合并

```
# 后端同学
git add .
git commit -m "feat: 完成车辆和品牌CRUD Service层+Controller"
git push origin feature/backend-car-crud

# 前端同学
git add .
git commit -m "feat: 完成车辆管理增删改查，品牌下拉动态加载"
git push origin feature/frontend-car-crud

# 组长合并到dev
git checkout dev
git pull origin dev
git merge feature/backend-car-crud
git merge feature/frontend-car-crud
# 若有冲突，手动解决后提交
git push origin dev

# 所有同学拉取最新
git checkout dev
git pull origin dev
```

九、今日自检清单

- `CarService` 接口和 `CarServiceImpl` 实现类已创建

- `BrandService` 接口和 `BrandServiceImpl` 实现类已创建
- `CarController` 注入的是 `CarService`
- `BrandController` 注入的是 `BrandService`
- Service层包含完整的业务校验：
 - 品牌非空校验
 - 车型非空校验
 - 价格合法性校验 (>0)
 - 车型重复校验 (同一品牌下)
- Postman测试所有车辆接口通过
- Postman测试所有品牌接口通过
- 品牌下拉框从后端动态加载 (非硬编码)
- 前端新增、编辑、删除功能完整可用
- 所有代码已提交并合并到 `dev`

十、课后预习

- 第4天将开发试驾预约功能 (同样使用三层架构)
- 预习预约冲突校验的SQL写法: 如何判断同一车辆同一时间段是否已被预约
- 思考: `@Transactional` 在Service层如何保证数据一致性